3.3.1 Technology Stack Revisited

Uploaded material

File	Content
1_FamPlan_System_Description.pdf	SRS v1.5 Sections 1–2 (purpose, scope, product functions, users, operating environment, constraints), 4 pages
2_Class_Diagram_A_Entities.png	Updated entity model: 27 entity classes and 11 enumerations, each entity tagged with its persistence decision
3_Class_Diagram_B_Boundary_Control.png	Boundary and control classes for the five functional subsystems
4_Class_Diagram_D_Design_Layer.png	New design layer: start-up class, containers, persistence (Prisma/PostgreSQL), access control, Adapter/Strategy/Observer roles

Method

The rerun used a fresh session of OpenAI GPT-6.1 Sol (in the Codex app) on 16 October 2026, with the four files attached and no follow-up messages. Two inputs differ from Lab 2 by necessity: (1) the exact Lab 2 prompt was not saved, so the course's standard wording from Lab 2 Section 3.4.2 was used unchanged; (2) the project moved from HDB housing to family-neighbourhood selection on 12 September, so the current system description (SRS v1.5) replaced the out-of-scope Lab 1 description. The Lab 2 answer used for comparison is *Lab2/FamPlan_Technology_Stack_Recommendation.pdf* (16 September 2026). Our technology preferences did not change between the two runs.

Text prompt used

"Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team."

AI Textual Output (verbatim)

GPT-6.1 Sol's complete reply. Link targets are shown as plain text; the four "[source]" lines are the citation markers the tool added for the attached files.

I recommend **React + TypeScript + Vite for the frontend, Node.js + Express for the backend, and PostgreSQL + Prisma for storage**, organised as one application with separate feature modules.

This suits an entry-level team because TypeScript works across both frontend and backend, while the architecture closely follows your existing controllers, entities, middleware, and

data-source adapters. Diagram D already identifies Express, Prisma, PostgreSQL, and Zod, so this recommendation also keeps implementation aligned with your design.

Area	Recommended technology	Why it fits FamPlan
User interface	**React + TypeScript + Vite**, with plain CSS	Reusable components for search, preferences, neighbourhood details, comparisons, and shortlists. Vite provides a React–TypeScript starter. [Documentation: vite.dev/guide]
Backend	**Node.js, supported LTS release + Express**	Maps naturally to your controllers and access-control middleware. Keep account, discovery, scoring, and synchronisation logic in separate modules within one server. [Documentation: expressjs.com]
Database	**PostgreSQL + Prisma ORM**	Your accounts, preferences, shortlists, facilities, and dataset history have clear relationships. Transactions support activating a validated snapshot without leaving partially updated data. [Documentation: prisma.io/docs/orm/fundamentals/transactions]
Input validation	**Zod**	Share validation schemas between client and server for weights, destinations, notes, and search filters; always validate again on the server. [Documentation: zod.dev]
Maps and routes	**Leaflet + OneMap APIs**	Leaflet displays interactive maps; OneMap supplies Singapore geocoding and supported routing. Keep provider credentials and authenticated API calls on the server. [leafletjs.com, onemap.gov.sg/apidocs]
Automatic updates	**node-cron** within an always-running server	Fits SyncScheduler for the initial deployment. Check overdue updates on startup and prevent overlapping runs. A sleeping host needs an external scheduler. [Documentation: nodecron.com/api-reference.html]
Testing	**Vitest + Playwright**	Test scoring, validation, and snapshot handling with Vitest; test registration, comparison, and shortlist journeys in the browser with Playwright. [vitest.dev/guide, playwright.dev/docs/intro]
Deployment	**One Node.js service + managed PostgreSQL**	Express can serve the built frontend and API under the same origin, simplifying cookies, deployment, and troubleshooting.

For authentication, follow the session model already shown: server-stored sessions, HttpOnly, Secure, SameSite cookies, a maintained password-hashing library, and server-side role checks. Use an email delivery provider for password resets and protect state-changing requests against CSRF.

For neighbourhood calculations, start with the in-memory index and geographic utilities in Diagram D. Calculate scores on demand, persist the entities marked as persistent, and keep

source dates and unavailable-data states visible. Introduce a spatial database extension only if measured performance requires it.

Keep the first version small: one repository, one backend, and one database. Microservices, Redis, Kubernetes, and machine-learning scoring would add learning and operating work without a demonstrated need in these materials.

The recommendation draws on the supplied system description and three diagrams:

[source] 1_FamPlan_System_Description.pdf
[source] 2_Class_Diagram_A_Entities.png
[source] 3_Class_Diagram_B_Boundary_Control.png
[source] 4_Class_Diagram_D_Design_Layer.png

3.3.2 Comparison with the Lab 2 Recommendation

How similar are the recommendations?

Mostly the same. Both answers recommend the same core stack (TypeScript, React with Vite, Node.js with Express, PostgreSQL with Prisma, Leaflet with OneMap and node-cron) as one modular application, and both advise against microservices, Redis and Kubernetes. The differences are in the level of detail, and most of them follow directly from what we added to the class diagrams.

Area	Lab 2 answer (16 Sep)	Lab 3 rerun (16 Oct)	Result
Language and frontend	TypeScript; React with Vite	TypeScript; React + Vite, with plain CSS	Same
Backend	Node.js + Express; one modular application	Node.js (LTS) + Express; one application with feature modules	Same
Database	PostgreSQL + Prisma	PostgreSQL + Prisma, using transactions to activate validated snapshots	Same, more detail
Maps and routes	Leaflet + OneMap	Leaflet + OneMap; credentials kept on the server	Same, more detail
Automatic updates	node-cron in the Express backend	node-cron; check overdue updates on startup, prevent overlapping runs, needs an always-running host	Same, more detail
Validation and security	Zod; server-side sessions; Argon2id; role checks	Zod; server-side sessions with HttpOnly/Secure/SameSite cookies; role checks; CSRF protection	Mostly the same
Neighbourhood search data	Indexed queries against validated data in PostgreSQL	In-memory index and geographic utilities (from Diagram D); persist only entities marked persistent	Changed
Testing	Vitest + Supertest + Playwright	Vitest + Playwright	Changed (Supertest dropped)
Deployment	Express serves the built React app; one database	One Node.js service + managed PostgreSQL, same origin	Same, more detail
Technologies to avoid	Microservices, Kubernetes, Redis, GraphQL, MongoDB, Elasticsearch, Next.js, NestJS, queues, ML	Microservices, Redis, Kubernetes, ML scoring	Same

(1) Differences explained by the updated class diagrams

Difference in the AI's output	Change in our input class diagrams that explains it
In-memory index instead of database queries for search	Diagram D adds **NeighbourhoodIndex**, a container class holding the neighbourhoods and facilities in memory, and **GeoUtils** for distance calculations. The Lab 2 analysis diagram had no such class, so the earlier answer assumed indexed PostgreSQL queries. The AI now says to "start with the in-memory index and geographic utilities in Diagram D".
Scores calculated on demand; only "persistent" entities stored	Diagram A now tags every entity **{persistent}**, **{transient}**, **{derived}** or **{constant}**. The AI repeats this decision almost word for word: "calculate scores on demand, persist the entities marked as persistent".
Cookie flags and CSRF protection	Diagram D adds **SessionAuthMiddleware** with an httpOnly, sameSite session cookie and requireUser / requireAdmin guards. The AI builds on this "session model already shown" and adds CSRF protection for state-changing requests.
Scheduler operating advice (startup check, no overlapping runs, always-running host)	Diagram D adds the start-up class **ServerBootstrap**, whose main() starts the scheduler and rebuilds the index, linked to SyncScheduler and DataSyncController. With the start-up path visible, the AI comments on what happens at startup and how the host must run.
Supertest no longer mentioned	This difference is not explained by the diagrams. It looks like ordinary variation between two AI answers; the team still uses Supertest for API tests.

Overall, the Lab 2 input was an *analysis* model (boundary, control and entity classes only), while the Lab 3 input is a *design* model that names infrastructure classes. The AI used those names directly. This also limits how much the rerun tells us: because Diagram D already names Express, Prisma, PostgreSQL and Zod, the AI partly read our own choices back to us, as it says itself ("Diagram D already identifies Express, Prisma, PostgreSQL, and Zod"). The agreement on the core stack is therefore weaker evidence than it looks. The independent value is in the operational advice, which we checked against the prototype: it already checks for overdue sources at startup (*needsStartupSync()* in scheduler.ts), never runs two synchronisations at once (*syncAll()* in pipeline.ts reuses the run in progress), and rejects state-changing requests that are not JSON as its CSRF defence (*requireJsonForMutations* in app.ts). The remaining point, that node-cron needs an always-running host, is a constraint we must respect when choosing where to deploy for the demo.

(2) Change of language or framework

Not applicable. We kept the same language and frameworks as in Lab 2, so no follow-up question about a technology change was needed.

3.3.3 AI Application Skeleton

Text prompt used

"Using our FamPlan design class diagram based on the Boundary-Control-Entity (BCE) pattern in TypeScript, generate a minimal skeleton folder hierarchy for the project. The structure must consist of empty files containing only // TODO comments indicating class responsibilities and method stubs. Do not write any implementation code, logic, mock returns, or library imports inside the files. Package the generated structure into a downloadable ai_skeleton.zip archive and output the corresponding directory tree in text."

AI Textual Output (Directory Tree Representation)

Please refer to ai_skeleton.zip uploaded in the github repository for the application skeleton

```
server/
├── boundaries/
│   ├── AccountBoundary.ts
│   ├── AdminMonitoringBoundary.ts
│   ├── DecisionSupportBoundary.ts
│   ├── DiscoveryBoundary.ts
│   ├── MapRouteBoundary.ts
│   ├── NeighbourhoodBoundary.ts
│   └── PreferenceBoundary.ts
├── controllers/
│   ├── AccountController.ts
│   ├── AdminMonitoringController.ts
│   ├── ComparisonController.ts
│   ├── DataSyncController.ts
│   ├── DiscoveryController.ts
│   ├── NeighbourhoodController.ts
│   ├── PreferenceController.ts
│   ├── RouteController.ts
│   ├── ScoringController.ts
│   ├── ShortlistController.ts
│   └── SyncScheduler.ts
├── entities/
│   ├── Facility.ts
│   ├── FamilyPreferenceProfile.ts
│   ├── FamilySuitabilityScore.ts
│   ├── NeighbourhoodProfile.ts
│   ├── SearchQuery.ts
│   ├── ShortlistEntry.ts
│   └── UserAccount.ts
└── external/
    ├── OneMapBoundary.ts
    └── PublicDataSourceBoundary.ts
```

Critical Evaluation of AI Output

- **Agreement with Choices**:
  The AI followed our BCE class diagram accurately, cleanly separating classes into boundaries/, controllers/, entities/, and external/ directories.
- **Identified Omissions:**
  It lumped all boundary classes into the server/ directory instead of separating client-side React UI boundaries from Express API routes. It also omitted shared TypeScript interfaces and database/ORM configuration (Prisma), which had to be added manually during prototype development.
- **Constraint Adherence:**
  The AI strictly adhered to instructions. No executable logic, mock return statements, or library imports were generated; all files contain solely // TODO comments and stub signatures.